

SWYP 앱 6기 9팀 · 백엔드 포트폴리오

FINDS

사진 미션·회고 앱 | 2026.08 ~ 진행 중

서버가 멈췄을 때는 데이터부터 지키고, 앱과 서버의 흐름이 어긋났을 때는 검증을 우회하지 않는 API로 풀고, 콘텐츠를 바꿀 때는 기존 기록의 연결을 보존했습니다.

이동영

백엔드 개발 · 앱-백엔드 연동 조율 (개발 리더)

이 자료에서 다루는 것

- 01 메모리 부족 서버 장애 대응
- 02 게스트 미션 이어하기 API (#54)
- 03 데이터를 보존한 미션 콘텐츠 전환 (#50)
- 04 운영 검증과 추가 기여 (#60 외)
- 05 KPI 정의를 계측 요구사항으로

팀 구성

PM 2 · 디자이너 3 · 앱 1 · 백엔드 3

근거 표시

- 코드** 저장소 코드·Git 이력에서 직접 확인
- 운영** 운영 기록·당시 테스트 실행 보고
- 미확인** 아직 검증되지 않은 사항

PROJECT 프로젝트·역할·구조

핵심 사용자 흐름

사진 취미를 막 시작한 사용자를 위한 앱입니다. 평가·경쟁 대신 하루 최대 3개 미션 중 하나를 골라 촬영하거나 갤러리에서 올리고, 짧은 회고로 자신의 기록을 쌓습니다.

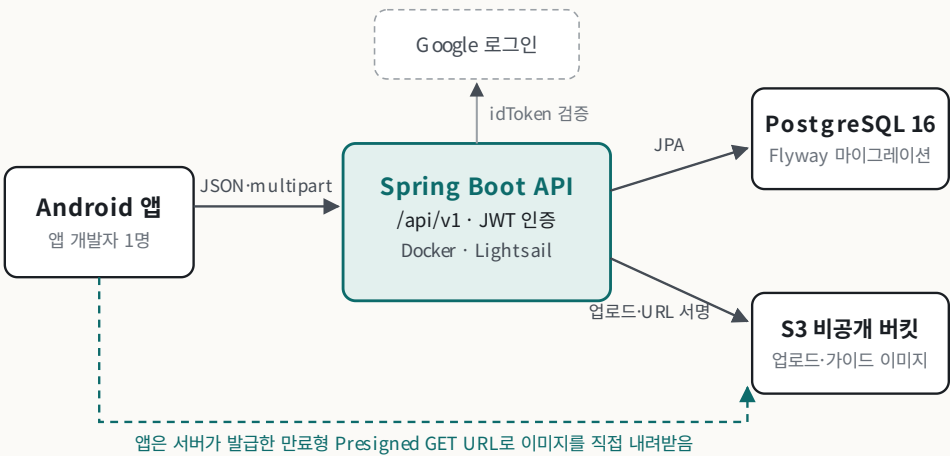
미션 탐색·선택 → 촬영/갤러리 업로드 → 짧은 회고 → 완료 기록

기술 스택 코드

언어·프레임워크	Java 17 · Spring Boot 4.1.0 · Gradle 9.5.1
데이터	Spring Data JPA · PostgreSQL 16 · Flyway (V1-V15)
이미지	AWS SDK v2 S3 · 비공개 버킷 + Presigned URL
테스트	JUnit 5 · Testcontainers (PostgreSQL 16)
로컬 환경	Docker Compose (개발용 PostgreSQL)
운영	AWS Lightsail · Docker 운영

내 역할

백엔드 3명 중 한 명으로 이미지·완료 기록·미션 콘텐츠 영역을 구현하고, 서버 배포·운영 확인과 앱 개발자와의 API 계약 조율을 맡았습니다. 인증·회원·후보 발급은 백엔드 팀원이 구현했습니다.



코드 기준 구조도. 운영 DockerCompose 파일은 서버에서 Git 미추적으로 관리되어 저장소에 없습니다.

구현 범위 구분 코드

구분	범위 (커밋 작성자 기준 + 사용자 확인)
본인 구현	이미지 업로드·검증, Presigned URL, 완료 기록 생성·동시성, 기록 목록·상세·최근 조회, 게스트 이어하기 (#54), 미션 콘텐츠 전환 (#50), 회고 300자 (#60)
팀원 구현	Google 로그인·토큰 재발급·로그아웃, 미션 도메인·후보 발급, 가입 동의, 회원탈퇴, 프로필 저장 (#57), 404 처리 (#58)
본인 운영	서버 장애 대응, 배포·기동 확인, PR 리뷰, 앱과의 API 계약 정리

CASE 01 · 운영 메모리 부족 서버 장애 — 증상과 진단

512MB 인스턴스에서 API 응답이 느려지고 연결이 끊겼습니다. 애플리케이션 버그를 찾기 전에 호스트 자원 상태부터 확인했습니다.

관찰한 사실 운영

영역	관찰 내용
API 응답	요청 응답 지연, 연결 초기화(connection reset) 발생
메모리·디스크	스왑 입출력이 심하고 I/O 대기가 높게 유지됨
Docker	<code>docker stop</code> 까지 지연되어, 바깥에 건 <code>timeout</code> 이 종료 코드 124(시간 초과)로 끝남
앱 컨테이너	종료 코드 137로 종료 — 프로세스가 SIGKILL로 끝났다는 의미

판단

스왑과 I/O 대기가 함께 높았고, 앱 요청뿐 아니라 **Docker 관리 명령까지 느려졌습니다**. 특정 API 로직이 아니라 호스트 전체가 메모리 부족으로 스왑에 의존하며 디스크 I/O가 포화된 상태로 판단했습니다. 따라서 코드 수정보다 **메모리 용량 확보**가 우선이라고 결론 내렸습니다.

단정하지 않은 것

불확실성

- 종료 코드 137은 SIGKILL을 뜻할 뿐입니다. 커널 OOM Killer일 수도, 중지 시간 초과 뒤의 강제 종료일 수도 있습니다.
- 컨테이너의 `OOMKilled` 값을 확인하지 않았으므로 “OOM 확정”으로 보지 않습니다. 미확인
- 프로세스별 메모리 점유 분해 수치는 이 자료에 포함하지 않았습니다.

진단 순서에서 지킨 원칙

- 관리 명령조차 느린 상태에서는 재시작을 반복하기 전에 **데이터를 먼저 보호**합니다.
- 증상(지연·137)과 원인(메모리 압박)을 분리해 기록하고, 확인한 범위만 결론에 씁니다.
- 운영 Compose·환경 파일이 서버에만 있다는 점을 고려해, 디스크를 통째로 옮길 수 있는 방법을 택합니다 → 다음 장.

메모리 부족 서버 장애 — 조치와 확인 결과

조치 순서 운영

1 상태 진단

메모리·스왑·I/O·컨테이너 종료 상태 확인

2 DB 논리 백업

`pg_dump` 성공, `pg_restore --list` 로 아카이브 목차가 읽히는지 확인

3 스냅샷 기반 2GB 인스턴스 이전

서버에만 있던 운영 설정 파일까지 디스크째 이전

4 JVM 메모리 옵션 조정

늘어난 메모리 안에서 앱 프로세스 상한 재설정

5 고정 IP·방화벽 설정

새 인스턴스에 고정 IP 연결, 방화벽 규칙 설정

6 앱 재기동·응답 확인

메모리·스왑 수치와 ping 응답 확인

이전 후 확인한 수치 운영

≈ 1.9 GiB

전체 메모리 (free -h)

≈ 1.1 GiB

사용 가능 메모리

0

swap in/out (vmstat 후속 샘플)

200

GET /api/v1/ping

왜 이 순서였나

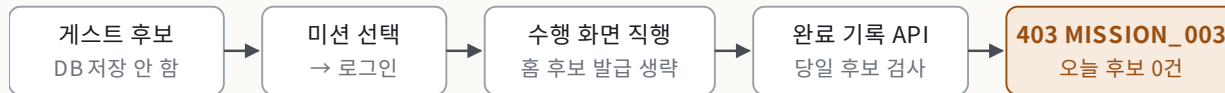
- 백업이 먼저 — 스왑 압박 상태에서 인스턴스 작업이 실패하면 DB를 잃을 수 있어, 이전 작업 전에 복구 수단부터 확보했습니다.
- 새로 구성하지 않고 스냅샷 — 운영 Compose·환경 파일이 Git에 없어서, 새 서버를 손으로 다시 만들면 설정 누락 위험이 컸습니다.
- 용량 확보 후 설정 조정 — 메모리를 먼저 늘린 뒤, 그 범위 안에서 JVM 메모리 옵션을 맞췄습니다.

검증 한계

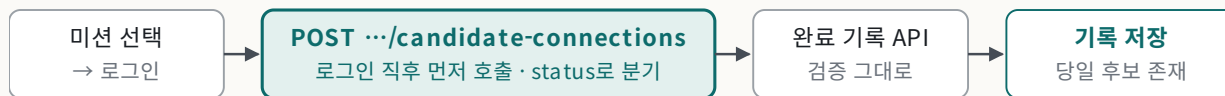
- 관찰 시점의 운영 확인입니다. 부하 테스트나 장기 안정성 검증은 아닙니다. 미확인
- 백업은 목차 확인까지이며, 실제 복원 테스트는 하지 않았습니다. 미확인
- 응답 속도 개선을 같은 정량 비교 수치는 측정하지 않았습니다.

게스트 선택 미션의 로그인 후 제출 실패 — 문제와 설계

기존 흐름 — 실패



개선 흐름 — 후보를 먼저 정당하게 만든다



장애 조사 (개인 식별값은 공개 자료에서 제외)

① 실패 요청의 업로드 이미지 소유자 확인 → ② 해당 사용자의 당일 mission_candidates 조회: 0건
→ 데이터 오류가 아니라 **앱의 화면 흐름과 서버 인가 조건이 어긋난 것**으로 판단

완료 API의 검증을 완화하지 않은 이유

완료 기록 API는 **당일 후보 여부·당일 중복 완료·이미지 소유권**을 검사하는 인가 지점입니다. 여기서 후보 검사를 빼면 어떤 미션이든 완료할 수 있게 됩니다. 그래서 검증은 그대로 두고, 로그인 직후 **선택한 미션을 오늘 후보로 연결하는 단계**를 앞에 추가했습니다. [코드](#)

세 가지 상태로 응답 [코드](#)

오늘 후보 상태	처리
없음	CONNECTED — 선택 미션을 1번으로, 나머지는 기존 발급 규칙(최근 48시간 노출 제외·가중치 선정)으로 최대 3개 발급
선택 미션 포함	ALREADY_CANDIDATE — 기존 세트 그대로. 발급 후 비활성화된 미션도 이어서 수행 가능
선택 미션 미포함	REJECTED_CANDIDATES_ALREADY_FIXED — 기존 세트를 바꾸지 않고 거부

세 결과 모두 HTTP 200입니다. `success: true`가 연결 성공을 뜻하지 않으므로 앱은 반드시 `data.status`로 분기해야 한다고 API 문서에 명시했습니다. 인증 실패·ID 형식 오류·존재하지 않거나 비활성인 미션은 별도 4xx로 응답합니다.

앱과 맞춘 호출 계약

- 로그인 직후 **홈 후보 발급보다 연결 API를 먼저** 호출합니다. 홈 발급이 먼저 끝나면 다른 3개가 확정되어 연결이 거부될 수 있기 때문입니다.
- 거부 상태에서는 기존 오늘 후보를 응답에 함께 내려주어, 앱이 홈 화면으로 자연스럽게 안내할 수 있게 했습니다.
- 이 API가 **증명하지 않는 것**(게스트 화면에서 실제로 봤는지)을 문서에 따로 적었습니다 → 다음 장.

게스트 선택 미션 이어하기 — 동시성과 검증

홈 후보 발급과 같은 잠금으로 직렬화 [코드](#)

```
// 설명용 의사코드 — 직접 작성, 실제 서비스 코드 발췌 아님
연결요청(인증 사용자, 선택 미션):
  사용자 행 잠금          // SELECT ... FOR UPDATE, 홈 후보 발급과 동일
  기준시각 = 시계.지금()   // 잠금 획득 후 한 번만 읽음
  오늘 = KST 날짜(기준시각)
  오늘후보 = 후보 조회(사용자, 오늘)
  만약 오늘후보가 있으면:
    선택 미션이 포함 → ALREADY_CANDIDATE      // 세트 유지
    포함되지 않음 → REJECTED_CANDIDATES_ALREADY_FIXED
  선택 미션 존재·활성 확인    // 실패 시 MISSION_001
  후보 = [선택 미션] + 기존 발급 규칙으로 고른 나머지
  저장 후 → CONNECTED
```

실제 서비스 코드는 팀의 비공개 저장소에 있으며 이 자료에는 포함하지 않았습니다.

- 홈 후보 발급(`issueUserCandidates`)도 같은 사용자 행을 먼저 잠급니다. 두 요청이 동시에 와도 **먼저 잠근 쪽만 오늘 후보 세트를 확정**하고, 뒤의 요청은 확정된 세트를 보고 판정합니다.
- 잠금을 얻은 뒤 시각을 한 번만 읽어, 한 요청 안에서 “오늘”이 바뀌지 않게 했습니다.
- 나머지 슬롯은 기존 발급 로직을 그대로 재사용해 규칙이 두 갈래로 나뉘지 않게 했습니다.

현재 상태 — 백엔드 구현·배포 완료, 앱 연동 QA 미확인

배포 후 기존 앱으로 제출했을 때 MISSION_003이 다시 발생했습니다. 그 앱은 아직 연결 API를 호출하지 않는 버전이었습니다. 연결 API를 적용한 앱 버전의 종단 간 QA 완료는 마지막 확인 시점까지 보고받지 못했습니다. [미확인](#)

검증 범위

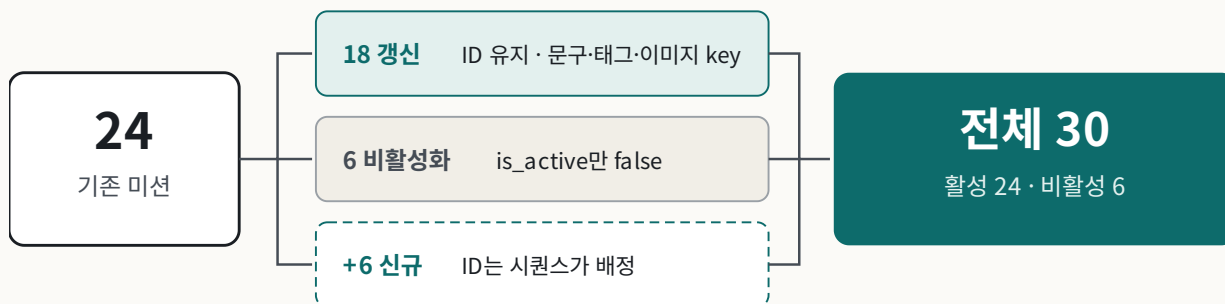
항목	내용
동시성	발급·연결 동시 요청 시 후보가 정확히 3개로 유지, 서로 다른 미션 동시 연결 시 한 요청만 성공 — 실제 PostgreSQL 16 컨테이너 코드
종단 흐름	연결 후 완료 기록 생성 성공, 연결→발급 순서 시 같은 세트, 발급→세트 밖 연결 거부, 사용자 간 격리, 전날 후보 무시 코드
실행 결과	개발 시점 전체 314건, 실패·오류·스킵 0 운영
배포	e6cdb17 기반 배포·기동 확인 운영

절충과 남은 위험

- **정책적 절충:** 게스트 노출을 증명하는 토큰이 없어, 오늘 후보가 없는 인증 사용자가 활성 미션 1개를 직접 고를 수 있습니다. 문서에 명시했습니다.
- 미션 활성 확인과 저장 사이의 비활성화 경쟁은 미션 행 잠금이 없어 원자적으로 막지 않습니다.
- 자정 전후·잠금 대기 중 날짜 변경은 직접 테스트하지 않았습니다. 고정 시각을 주입한 Clock으로 전날 후보 케이스만 검증했습니다.

기존 기록을 보존한 최종 미션 콘텐츠 전환

최종 기획 콘텐츠로 교체하면서, 이미 쌓인 후보·완료 기록이 가리키는 미션 ID를 하나도 지우거나 다른 미션으로 재사용하지 않았습니다.



왜 삭제·재사용을 하지 않았나

`mission_candidates`와 `mission_records`가 미션 ID를 참조합니다. 삭제하면 FK에 막히거나 기록이 끊기고, ID를 다른 미션에 재사용하면 과거 기록이 **전혀 다른 미션을 가리키게** 됩니다. 그래서 갱신·비활성화·신규 추가 세 가지만 사용했습니다.

마이그레이션 설계 [코드](#)

- **V13** — `reference_image_key`를 nullable로 추가하고 기존 `reference_image_url`은 그대로 둬. 배포 중인 구버전 앱 인스턴스는 새 컬럼을 매핑하지 않아 영향이 없음.
- **V14** — 활성 미션 CHECK 제약을 “URL 또는 key”로 바꾼 뒤 콘텐츠 반영. 신규 6개는 key만 가짐. 테스트용 H2와 같이 동작하도록 `ON CONFLICT` 대신 `NOT EXISTS` 사용.

응답 시 이미지 해석 규칙 [코드](#)

- `MissionReferenceImageResolver` 한곳에서 처리: key가 있으면 Presigned URL을 발급하고, 없을 때만 기존 URL을 반환합니다. 두 값을 섞지 않습니다.
- 서명이 실패하면 예외를 그대로 전파합니다. **더미 URL로 감추지 않습니다.**

비활성 미션 정책 [코드](#)

- 이미 발급된 **본인의 당일 후보**는 비활성화 후에도 재조회·상세·완료 허용
- 신규 후보 선정과 게스트 후보에서는 제외, 다른 사용자의 상세·완료는 거부

운영 적용 확인 [운영](#)

Flyway V13·V14 성공 · 전체 30 / 활성 24 / 비활성 6 · 이미지 key가 있는 활성 미션 24개

알고 있는 한계

- 기록 상세는 완료 당시 스냅샷이 아니라 **현재 미션 제목·태그**를 읽습니다. 갱신된 18개 미션의 과거 기록 표시도 바뀝니다.
- 코드만 이전 버전으로 되돌리면 구버전은 URL 필드만 읽으므로 신규 6개 미션의 이미지 URL이 null이 됩니다. 롤백에는 데이터 대응이 함께 필요합니다.

OPERATIONS 운영 검증과 추가 기여

A 가이드 이미지 403 진단

#50 배포 후 가이드 이미지의 Presigned URL은 정상 발급되는데 다운로드가 403이었습니다. [운영](#)

- **판단:** Presigned URL은 SDK가 로컬에서 서명만 계산하므로 생성 성공이 권한을 보장하지 않습니다. 권한은 다운로드 시점에 서명 주체 기준으로 평가됩니다.
- **원인:** 실제 응답을 조사한 결과, 앱 IAM 주체에 가이드 이미지 접두사의 `s3:GetObject` 허용이 없었습니다.
- **조치:** 해당 접두사에 한정한 읽기 권한만 추가했습니다. 버킷 공개는 하지 않았습니다.
- **확인:** 이미지 1건 HTTP 200, `image/jpeg` 다운로드

[미확인](#) 24개 전체 이미지 다운로드 검증하지 않았습니다.

B 회고 300자 — 계약과 DB 제약 일치 (#60)

기획은 세 항목 각 300자인데, 서버는 한 줄 평 50자·잘한 점/어려운 점 100자로 제한하고 있었습니다.

- **조치:** 요청 DTO `@Size(max = 300)`, 엔티티 `length = 300`, 신규 V15 로 `VARCHAR(300)` 을 함께 맞춤. 적용된 기존 마이그레이션은 수정하지 않음 [코드](#)
- **검증:** 컨트롤러 테스트 26건(299·300자 허용, 301자 거부). V14 데이터가 있는 DB에 V15 적용·기존 데이터 보존과 신규 DB 전체 마이그레이션을 PostgreSQL 테스트 11건으로 확인 [운영](#)
- **배포:** `918b5c2`, ping 200, V15 성공, 세 컬럼 길이 300 확인 [운영](#)

전체 337건 실행은 Docker 없이 262건 성공·75건 스킵이었으므로 “전체 통과”로 보지 않습니다. 길이는 `String.length()` 기준이라 이모지가 섞이면 체감 글자 수와 다를 수 있습니다. 실제 앱 300자 제출 QA는 [미확인](#)

C 그 밖의 기여와 협업

- **#52 이미지 비율:** 3:4 정수 일치 검사를 목표 비율 대비 상대 오차 0.5% 이하 허용으로 변경. 회전·크롭·리사이즈는 넣지 않고 경계값 테스트와 거부 시 크기 로그를 추가 [코드](#). 당시 293건 통과·배포 [운영](#). 실제 실패 사진 해결 여부는 [미확인](#)
- **#49 로컬 개발 환경:** PostgreSQL 16 Compose, 환경변수 표, 포트 충돌 시 셸 환경변수 우선순위 주의사항까지 README에 정리 [코드](#)

팀원 구현과 내 역할 구분

Google 프로필 저장·로그인 갱신(PR #57)과 없는 경로 404 처리(PR #58)는 **백엔드 팀원이 구현**했습니다. 저는 코드 리뷰, 배포, 배포 후 동작 확인을 맡았습니다.

앱과의 계약은 `docs/api` 에 상태값·호출 순서·오류 코드, 그리고 “이 API가 증명하지 않는 것”까지 적어 공유했습니다.

KPI 정의를 실제 계측·API 요구사항으로 구체화

PM의 KPI·이벤트 문서를 실제 앱 화면 흐름과 서버 API 처리 단위에 맞춰 검토했습니다. 아래는 설계 검토와 협의 결과이며, 수집·분석은 아직 시작 전입니다.

1 발견한 문제

- 사진 업로드 실패가 완료 실패 이벤트에서 빠져 있음
- 오류 유형이 앱이 실제로 구분할 수 없는 기준으로 정의됨
- 노출과 선택의 집계 단위가 달라 전환율이 왜곡될 수 있음
- 미션 시작 시점이 버튼 클릭 기준이라 실제 수행과 어긋남
- 게스트 이어하기는 선택 없이 시작부터 기록될 수 있음
- 회고 작성량 구간이 300자 정책과 맞지 않음

2 제안한 기준

- 업로드 실패도 완료 실패에 포함
- 오류 유형은 업로드·최종 제출·네트워크로 나누고, 재시도는 각각 별도 시도로 집계
- 노출·선택 모두 **앱 인스턴스·미션·KST 날짜** 기준으로 각각 중복 제거
- `mission_start` = 실제 수행 화면 진입
- `mission_complete` = 최종 저장 API 성공 응답 이후
- 게스트 이어하기의 select 누락을 분석 예외로 명시
- 작성량 구간 갱신, 회고 원문은 전송하지 않음

3 PM과 정리된 방향

- 핵심 퍼널: `mission_select` → `mission_start` → `mission_complete`
- 첫 완료일 D0를 제외한 KST D1~D7 추가 완료 집계
- D7 관찰이 끝난 앱 인스턴스만 분모, QA 대상 제외
- MVP 분석 단위는 회원 계정이 아닌 앱 인스턴스
- D1~D7 집계·태그 매핑은 BigQuery 기준으로 진행 예정
- 필터 적용 이벤트는 MVP에서 제외

4 남은 구현

- 앱 이벤트 구현 — 앱 개발자 담당
- 분석 선택 동의의 저장·조회·변경·철회와 로그인 상태에 따른 수집 제어 — 앱 개발자와 연동 범위 정리
- 후속 이슈 #62 — 기존 코드 조사, 설계 보완, PM 정책 협의까지 진행(10.02 기준). 구현·배포 완료 보고 없음 **미확인**
- Daily Export 연결과 실제 적재 확인 — 후속 작업 **미확인**

대표 판단 — 노출과 선택은 같은 키로 세야 비교할 수 있다

노출을 화면이 그려질 때마다 세고 선택은 한 번만 센다면, 같은 사람이 홈을 여러 번 열수록 선택률이 실제보다 낮아집니다. 그래서 두 이벤트 모두 **앱 인스턴스·미션·KST 날짜** 조합으로 각각 중복 제거하자고 제안했고, 날짜 경계는 서버의 당일 후보 기준(KST)과 맞췄습니다.

기여 범위 — KPI·정책 문서 작성과 최종 기획 결정은 PM과의 공동 작업이고, 앱 이벤트 구현은 앱 개발자 담당입니다. 저는 기술적 모순·누락 검토와 서버 계약·연동 범위 조율에 기여했습니다.

지표 개선 수치나 분석 성과는 아직 없습니다. 근거는 비공개 팀 문서·대화이므로 원본 화면 대신 익명화한 요약으로 정리했습니다.

마지막 확인 기준 상태

사례	확인된 상태
서버 장애	2GB 이전 후 관찰 시점 swap in/out 0, ping 200 운영
#54 이어하기	백엔드 구현·배포 완료, 앱 연동 QA 미확인
#50 콘텐츠	V13·V14 적용, 30/24/6 확인, 가이드 이미지 1건 다운로드 확인 운영
#60 회고 300자	V15 적용·컬럼 300 확인, 앱 300자 제출 QA 미확인
KPI 협업	기준 합의, #62 설계·정책 협의 중 미확인

2026년 10월 10일 데모데이가 예정되어 있으며, 개발은 진행 중입니다.

일하면서 지킨 판단 기준

1. 인가 지점은 완화하지 않는다. 흐름이 어긋나면 앞단에서 정당한 상태를 만든다.
2. 데이터 변경은 참조 보존부터 정한다. ID를 지키고, 롤백 제약을 문서에 남긴다.
3. “생성됨”과 “동작함”을 구분한다. URL 서명과 다운로드 권한, 배포와 앱 QA, 테스트 존재와 실행 결과는 각각 확인한다.
4. 장애 때는 원인 단정보다 데이터 보호가 먼저다.
5. 입력 계약과 저장 제약은 함께 바꾼다. DTO만 늘리면 DB에서 다시 실패한다.

코드를 어떤 방식으로 작성했는지와 별개로, 문제 정의·설계 선택·리뷰·검증·배포와 운영 판단의 책임은 제게 있었습니다.

근거가 있는 다음 과제

- 앱 연동 QA — 연결 API 적용 앱으로 게스트→로그인→제출, 300자 제출을 중단 확인
- 요청 추적성 — 지금은 토큰 재발급 API에만 traceId 로그가 있습니다(#23). #60처럼 특정 실패 요청을 로그로 바로 찾을 수 있게 전체 요청으로 넓히기
- 배포 재현성 — 서버에만 있는 운영 Dockerfile·Compose를 비밀값을 뺀 형태로 버전 관리
- 백업 복원 리허설 — 목록 확인을 넘어 별도 DB로 실제 복원
- 메모리 관측 — 스왑 사용과 컨테이너 종료 원인 (OOMKilled)을 남기는 모니터링